

Name: Sadia Shafeeq

Roll No: BSAI23009

1. Project Overview

This project implements a custom indexed file system in C++ that simulates real-world file management operations. Unlike FAT-based systems, this design uses indexed allocation where each file maintains an index block containing direct pointers to its data blocks. This approach enables efficient random access and simplified metadata management.

System Specifications

- Directory Size: 128 entries
- Maximum File Name Length: 64 bytes
- Maximum File Size: 128 blocks (128 KB)
- Block Size: 1 KB (1024 bytes)
- Total Disk Size: 64 MB
- Allocation Method: Indexed Allocation
- Implementation Language: C++

2. Design Philosophy

The design prioritizes simplicity, efficiency, and educational clarity. Indexed allocation was selected over linked allocation because it provides direct access to file blocks, faster random I/O operations, and centralized file metadata without fragmentation.

3. System Architecture

The file system consists of a superblock, bitmap, directory table, index blocks, and data blocks. Frequently used metadata is cached in memory and synchronized to disk after each modification to ensure persistence.

Disk Layout

Block 0 : Superblock

Blocks 1-2: Bitmap

Blocks 3-9: Directory Entries

Blocks 10+: Index Blocks and Data Blocks

4. Implementation Details

Low-level block I/O is handled using ReadBlock and WriteBlock functions, which operate on fixed-size 1 KB blocks. Free space management is implemented using a bitmap, and first-fit allocation is used to locate free blocks.

5. File and Directory Operations

The system supports creating, deleting, reading, writing, and truncating files. Directory operations include creation, deletion, and listing of entries. Partition-level operations such as formatting and mounting are also implemented.

6. Challenges and Solutions

Key challenges included bitmap size handling, ensuring file persistence, index block management, and efficient block allocation. These were resolved through multi-block bitmap storage, explicit metadata synchronization, fixed-size index blocks, and linear bitmap scanning.

7. Compilation and Execution

Compilation Command:

```
g++ -std=c++11 -o filesystem filesystem.cpp
```

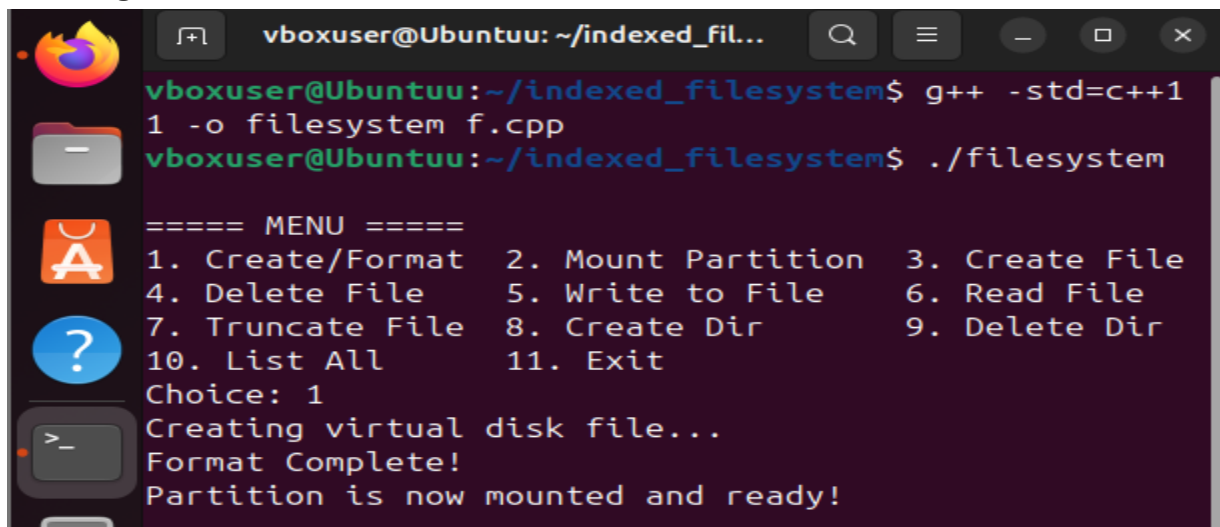
Execution Command:

```
./filesystem
```

8. Testing and Validation

The system was tested using multiple scenarios including file creation, data writing, reading, deletion, truncation, directory handling, and persistence across program executions.

Creating Partition:

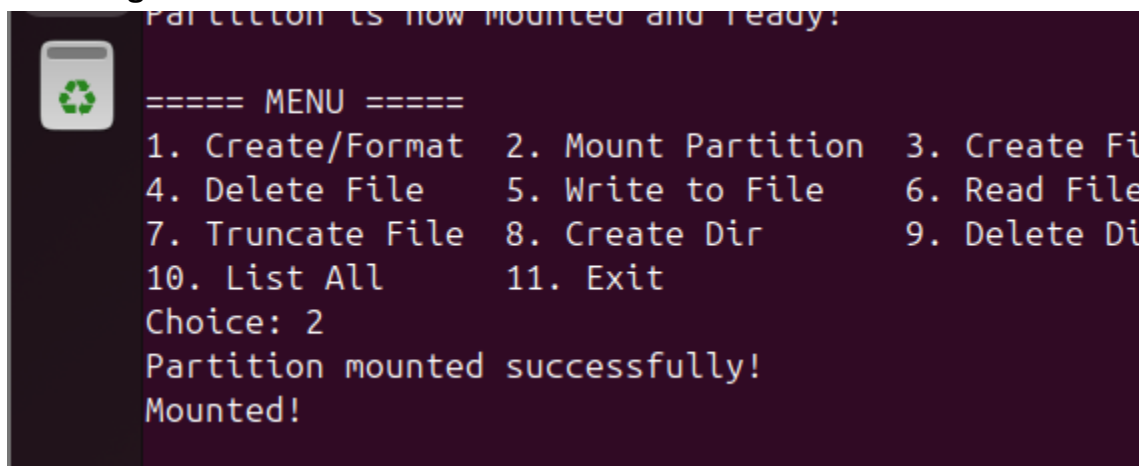


A terminal window titled 'vboxuser@Ubuntu: ~/indexed_fil...' shows the execution of a C++ program. The user enters 'g++ -std=c++11 -o filesystem f.cpp' and then './filesystem'. The program displays a menu with 11 options. The user selects option 1, 'Create/Format'. The program then outputs: 'Creating virtual disk file...', 'Format Complete!', and 'Partition is now mounted and ready!'.

```
vboxuser@Ubuntu:~/indexed_fil...$ g++ -std=c++11 -o filesystem f.cpp
vboxuser@Ubuntu:~/indexed_fil...$ ./filesystem

===== MENU =====
1. Create/Format    2. Mount Partition    3. Create File
4. Delete File      5. Write to File      6. Read File
7. Truncate File    8. Create Dir         9. Delete Dir
10. List All        11. Exit
Choice: 1
Creating virtual disk file...
Format Complete!
Partition is now mounted and ready!
```

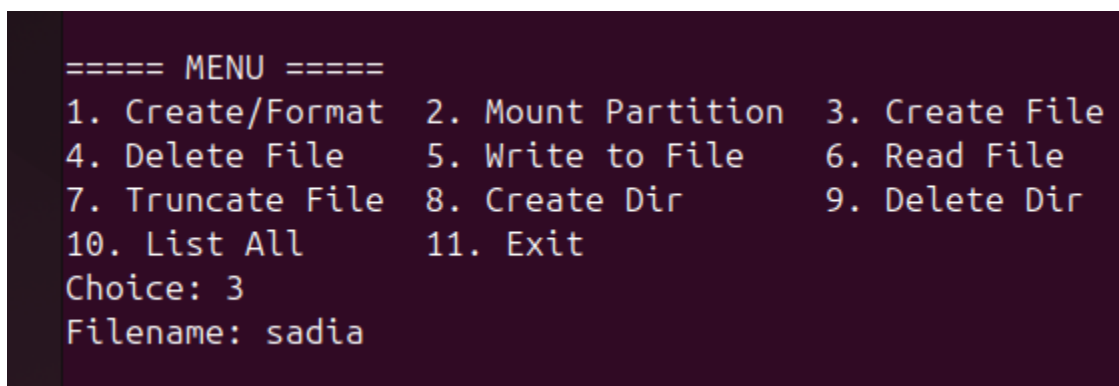
Mounting Partition:



A terminal window shows the execution of a C++ program. The user enters '2' to select 'Mount Partition'. The program outputs: 'Partition mounted successfully!' and 'Mounted!'.

```
===== MENU =====
1. Create/Format    2. Mount Partition    3. Create File
4. Delete File      5. Write to File      6. Read File
7. Truncate File    8. Create Dir         9. Delete Dir
10. List All        11. Exit
Choice: 2
Partition mounted successfully!
Mounted!
```

Creating File:



A terminal window shows the execution of a C++ program. The user enters '3' to select 'Create File'. The program prompts for a filename, and the user enters 'sadia'.

```
===== MENU =====
1. Create/Format    2. Mount Partition    3. Create File
4. Delete File      5. Write to File      6. Read File
7. Truncate File    8. Create Dir         9. Delete Dir
10. List All        11. Exit
Choice: 3
Filename: sadia
```

Deleting file:

```
===== MENU =====
1. Create/Format  2. Mount Partition  3. Create File
4. Delete File    5. Write to File   6. Read File
7. Truncate File  8. Create Dir       9. Delete Dir
10. List All      11. Exit
Choice: 4
Filename: sadia
```

Write to a File:

```
===== MENU =====
1. Create/Format  2. Mount Partition  3. Create File
4. Delete File    5. Write to File   6. Read File
7. Truncate File  8. Create Dir       9. Delete Dir
10. List All      11. Exit
Choice: 5
Filename: sadia
Data: hi, its sadia here
```

Read File:

```
===== MENU =====
1. Create/Format  2. Mount Partition  3. Create File
4. Delete File    5. Write to File   6. Read File
7. Truncate File  8. Create Dir       9. Delete Dir
10. List All      11. Exit
Choice: 6
Filename: sadia
Content: hi, its sadia here
```

Truncate File:

```
===== MENU =====  
1. Create/Format  2. Mount Partition  3. Create File  
4. Delete File    5. Write to File    6. Read File  
7. Truncate File  8. Create Dir       9. Delete Dir  
10. List All      11. Exit  
Choice: 7  
Filename: sadia
```

Create Dir:

```
===== MENU =====  
1. Create/Format  2. Mount Partition  3. Create File  
4. Delete File    5. Write to File    6. Read File  
7. Truncate File  8. Create Dir       9. Delete Dir  
10. List All      11. Exit  
Choice: 8  
Dir Name: ss
```

Delete Dir:

```
===== MENU =====  
1. Create/Format  2. Mount Partition  3. Create File  
4. Delete File    5. Write to File    6. Read File  
7. Truncate File  8. Create Dir       9. Delete Dir  
10. List All      11. Exit  
Choice: 9  
Dir Name: SS
```

3

List All:

Name	Type	Size (bytes)

ss	<DIR>	0
sadia	<FILE>	0
a	<FILE>	0
B	<FILE>	0
ASAS	<DIR>	0

Exit:

```
===== MENU =====
1. Create/Format  2. Mount Partition  3. Create File
4. Delete File    5. Write to File    6. Read File
7. Truncate File  8. Create Dir        9. Delete Dir
10. List All      11. Exit
Choice: 11
vboxuser@Ubuntuu:~/indexed_filesystem$
```

9. Learning Outcomes

This lab provided practical understanding of file system design, indexed allocation strategies, low-level disk I/O, metadata management, and systems programming in C++.

10. Conclusion

The indexed file system implementation fulfills all Lab 12 requirements and successfully demonstrates core operating system concepts related to file storage and allocation. The system is stable, efficient, and ready for academic evaluation.